

When Does LLM-Serving Scheduler Adaptation Matter? Action Opportunity and Causal Headroom in Production-Derived Replay

Soroush Vahidi

New Jersey Institute of Technology, Newark, NJ, USA

Abstract

Adaptive scheduling for large language model (LLM) serving pays off only if the live system reaches states where a different executable action exists and taking it improves performance. We measure it before any adaptive scheduler is designed. In a deterministic simulator replaying production-derived traces (Azure code, Azure conversation, BurstGPT), we count how often a six-policy portfolio proposes an action different from the single best scheduler’s and, on fresh non-overlapping windows, force one alternative action for one step before returning to the default. Under abundant resources, replay was action-null in about one million decision states. Capping key-value (KV) cache capacity or concurrent sequences created disagreement; scaling arrival rates up to eight times did not. In a preregistered analysis of 720 fresh disagreement states, 590 (81.9%) had positive one-step latency headroom; the state-weighted mean oracle headroom, the best available one-step latency reduction, was 2.00 ms (95% window-clustered confidence interval 0.19–3.55 ms). Post hoc analyses show that the sign is stable across weightings and leave-one-out analyses, but the magnitude is not: the median is 0.20 ms, and one of 36 windows holds 88% of the total headroom. A bounded vLLM probe shows that resource pressure changes scheduler behavior but does not validate the simulated latency effect. Adaptation opportunity is real but sparse, pressure-dependent, and generally modest; detecting it does not show that a deployed adaptive scheduler is worthwhile.

Keywords: LLM serving, request scheduling, performance evaluation, workload replay, resource contention, causal evaluation

1. Introduction

Adaptive scheduling in large language model (LLM) serving is usually motivated by heterogeneous prompts, decode demand, key-value (KV) cache residency, queueing, and deadlines. It can help only if the live system reaches states where a different executable action exists, such as another admission, batch, or request order, and only if taking that action improves the target metric by more than the cost of choosing it. How often this happens is rarely measured before a scheduler is designed.

Common evaluation practice does not answer this. Ranking policies on whole-trace metrics, or reporting the gap to a virtual best solver (VBS) as in algorithm portfolios [1, 2], shows that policies differ but aggregates thousands of decisions; it cannot say whether an alternative was available at a given state. Policies with different scores can issue the same batch, and an alternative that exists may not reduce latency. Recent systems address *how* to adapt [3, 4, 5, 6]. We ask the prior question: *when is there a different useful action to take, and how much is it worth?*

We answer it with a staged decomposition,

$$\begin{aligned} \text{resource pressure} &\longrightarrow P(D) \longrightarrow P(B_{\text{LAT}} \mid D) \\ &\longrightarrow \text{headroom magnitude} \longrightarrow \text{predictability and deployment.} \end{aligned} \tag{1}$$

Here D means that at least one canonical executable alternative differs from the default single best scheduler, and B_{LAT} that at least one such alternative lowers one-step mean latency relative to that default. These are distinct quantities: pressure can make policies disagree often without the disagreement being useful, and useful disagreement can be rare or small. The last arrow, whether a selector could exploit the opportunity, is not measured here.

This is a performance-evaluation question: at which operating points does scheduler choice have causal performance consequences? Answering it takes workload characterization, controlled resource interventions, replay-based simulation, and inference that respects the dependence among states from one trace window. Recent work in *Performance Evaluation* analyzes job assignment in machine-learning inference systems under accuracy constraints [7] and adaptive machine-learning training on serverless clouds [8]; our measurement-first view characterizes the region in which such mechanisms have something to choose between.

We replay production-derived traces (Azure 2023 code and conversation [9], BurstGPT [10]) in a deterministic simulator. Faithful replay of 60 windows gives the native baseline; one-axis interventions on active-sequence capacity, KV capacity, and arrival rate (1,080 conditions) locate where disagreement appears. On fresh non-overlapping windows we clone the simulator at each disagreement state, force one alternative action once, and return to the default. The primary endpoint, oracle latency headroom (the largest one-step mean-latency reduction among the alternatives, zero if none helps), was fixed in a preregistered protocol; the robustness analyses of the frozen results are post hoc. A bounded vLLM [11] probe checks the mechanism in a real engine.

Native replay is action-null under abundant resources. Capacity caps on active sequences or KV cache create disagreement; arrival-rate scaling up to eight times does not. Of 720 fresh disagreement states, 590 (81.9%; 589 excluding one floating-point-residue state) had positive headroom, with a state-weighted mean of 2.00 ms (95% window-clustered confidence interval 0.19–3.55 ms). The sign is robust: the mean stays positive under equal-window, equal-regime, and equal-workload weighting and when any single window or regime is removed. The magnitude is not uniform: the median is 0.20 ms, only 22.5% of states offer more than 2 ms, and one of 36 windows carries 88% of the total headroom. The vLLM probe shows that pressure alters queueing and scheduling behavior but does not validate the simulator’s latency magnitudes. Adaptation opportunity is thus real but sparse, pressure-dependent, and generally modest, and detecting it does not show that a deployed scheduler is worthwhile. The evidence covers three trace families, a six-policy portfolio, a simulator, and one bounded probe on a single model and GPU; the oracle is an opportunity bound, not a scheduler.

The study addresses five research questions, ordered along the decomposition.

RQ1 – Native opportunity. How often does the single best scheduler face a different executable action from the six-policy portfolio, in faithful replay of production-derived traces with abundant resources?

RQ2 – Pressure-induced opportunity. Which resource-pressure dimensions create action disagreement, and does the transition reproduce on untouched windows?

RQ3 – Causal value. At disagreement states, how often does forcing one

alternative action for one step reduce latency, and does the answer depend on the objective?

RQ4 – Magnitude and concentration. How large is the headroom, is it typical or concentrated, and how sensitive is it to weighting and to individual windows or regimes?

RQ5 – System correspondence. Does a bounded real vLLM deployment show the corresponding pressure and scheduling mechanism?

1.1. Contributions

This paper makes five contributions.

1. A staged action-opportunity evaluation that separates executable disagreement, beneficial disagreement, and headroom magnitude from policy-score diversity, with prevalence measured over all valid decision states and uncertainty quantified by trace window.
2. A characterization of where disagreement appears: native replay is action-null, and only capacity limits on active sequences or KV cache, not arrival-rate scaling, expose alternatives; the transition reproduces on untouched windows.
3. A preregistered one-step causal measurement on 720 fresh states, with latency as the primary endpoint because deadline goodput saturates at 1.0 and hides latency differences.
4. A robustness and concentration analysis showing that the sign of the headroom is robust but its magnitude is concentrated, so a mean reported without its distribution and cluster composition overstates the typical opportunity.
5. Bounded system evidence that resource pressure changes queueing and scheduling behavior in vLLM, without validating simulated latency magnitudes or a deployed selector.

Section 2 positions the work, and Section 3 defines the estimands and evidence design. Sections 4–6 report the native, pressure, and fresh causal results, Sections 7 and 8 cover objective sensitivity and the vLLM probe, and Section 9 discusses when adaptation is justified.

2. Related Work and Positioning

This section positions the paper against LLM-serving systems, workload studies, and algorithm-selection work. LLM-serving research spans model execution, prefill/decode scheduling, resource management, routing, caching, and cluster control. Orca and vLLM established iteration-level serving and paged KV management [12, 11]. Sarathi-Serve, DistServe, Splitwise, and FastServe study chunking, phase separation, and preemptive scheduling [13, 5, 14, 15]. Llumnix and SOLA adapt scheduling or state-aware service to dynamic conditions [3, 4]; Libra studies flexible request partitioning [16]; Mooncake and Strata emphasize KV-centric or hierarchical context caching [17, 18]; MorphServe changes model precision and KV capacity under pressure [19]; and DynamoLLM reconfigures cluster resources for performance and energy [20]. BurstGPT and Azure traces provide important workload evidence [10, 9]; the recent longitudinal study by Nixon et al. further emphasizes workload evolution, caching, and load-balancing over a year of production traffic [21].

ServeGen and OpenTela narrow any generic claim that workload realism or operational trace infrastructure is itself novel [22, 23]. LMetric is especially close: it combines a KV-aware indicator with an instance-load indicator, reports production-style workloads, and derives conditions under which its multiplication can fail [6]. These works answer how to build or evaluate better serving mechanisms. Our question is complementary: in a fixed workload/resource regime, do the default and alternatives expose different canonical actions at all, and does forcing one alternative action have default-relative causal value?

For this reason, systems such as ServeGen, Libra, Llumnix, MorphServe, Strata, LMetric, and OpenTela are related work and design context, not unimplemented baselines for a superiority claim. Reproducing each system would be necessary for a paper claiming that a new scheduler beats them. This paper instead asks whether an executable action opportunity and local default-relative headroom exist before building an adaptive selector. The single best scheduler/VBS-style quantities used below are non-deployable references for opportunity measurement, not schedulers proposed for production.

Table 1 records the scope honestly. A “partial” entry means that a work contains related evidence but does not use the paper’s canonical action or one-step estimand.

Our use of single best scheduler/VBS is rooted in algorithm portfolios

Table 1: Closest-work differentiation. We do not claim that other systems omit pressure, production-derived traces, or adaptive mechanisms; the distinction is the action-level and default-relative causal measurement question.

Work	Prod.- derived workloads	Pressure sweep	Proposes system	Canonical disagree- ment	One-step default in- tervention	Real system
ServeGen	Yes	No	Generator	No	No	Workload study
LMetric	Yes	Implicit	Yes	No	No	Yes
OpenTela	Operational traces	No	Control plane	No	No	Yes
Llumnix / SOLA	Selected regimes	Partial	Yes	No	No	System evalua- tion
Mooncake / Strata	Cache/resource regimes	Partial	Yes	No	No	Yes / opera- tional
This work	Yes	Yes	No	Yes, $P(D)$	Yes, simulator only	Bounded partial

and selection [1, 2, 24, 25]. The novelty claim is not the envelope itself, nor counterfactual reasoning in general. It is the narrow serving characterization in Eq. 1.

3. Methods

3.1. Evidence hierarchy and workload inventory

We report four evidence tiers. Tier 1 is native production-derived trace replay. Tier 2 is production-derived aggregate or window validation. Tier 3 is a controlled transformation of a real trace, such as one-axis resource caps. Tier 4 is synthetic or mechanistic workload construction. RQ1 uses Tier 1; RQ2 uses Tiers 2–3; fresh causal headroom uses a preregistered Tier-3 intervention over untouched windows; the real-vLLM result is a bounded system validation rather than a claim of production deployment.

The faithful corpus contains 20 windows each from Azure 2023 code, Azure 2023 conversation, and BurstGPT. Source order and token-length/arrival structure are retained. SLO and class overlays are deterministic simulator inputs; future actual output lengths are hidden from policies. The fresh confirmatory campaign uses 20 untouched windows per workload family for support mapping, with causal labeling limited to the frozen selected regimes. BurstGPT did not contribute a selected fresh causal disagreement regime, which is retained as an explicit heterogeneity limitation.

The historical six-policy portfolio contains full prefill, small-chunk prefill, estimated-service-first (ESTF), weighted fair sharing (WFS), least-laxity first (LLF), and a KV-constrained policy. six-policy is a fixed portfolio for a default-relative characterization, not an exhaustive set of 2026 schedulers.

3.2. Canonical action opportunity

Let $\mathcal{A}_6(s)$ be the set of canonical executable actions produced by the six portfolio policies at state s , after action deduplication. A canonical action is the simulator-executable admission, batch, and request ordering decision after infeasible or semantically identical variants have been collapsed. The default scheduler is single best scheduler, the fixed single best policy used as the canonical baseline for the frozen replay protocol; it is not refit per state. Let $a_{\text{singlebestscheduler}}(s)$ be single best scheduler’s action. We define

$$D(s) = \mathbf{1} [\exists a \in \mathcal{A}_6(s) : a \neq a_{\text{singlebestscheduler}}(s)]. \quad (2)$$

The opportunity prevalence is $P(D) = \sum_s D(s) / |\mathcal{S}_{\text{all}}|$, where the denominator is every valid single best scheduler decision state, not only states where policy scores differ. A policy ranking change that produces the same admission, batch, and request order is action collapse, not opportunity. An alternative action is any executable member of $\mathcal{A}_6(s)$ that is not equal to $a_{\text{singlebestscheduler}}(s)$.

This definition matters operationally. Whole-policy comparisons can show a latency or goodput difference accumulated over many decisions, but cannot say whether an alternative existed at a particular default state. Conversely, action diversity alone says nothing about usefulness.

3.3. One-step causal headroom

For every disagreement state s and canonical alternative a , we clone the state, force exactly one action, and return to single best scheduler for the continuation. Denote the resulting continuation value by $Q_{\text{singlebestscheduler}}(s, a)$.

This is a local counterfactual, not selector performance: it estimates the one-step effect of changing the current action while holding the subsequent default policy fixed.

For the fresh primary latency endpoint, continuation mean latency includes active, queued, and future requests belonging to the continuation, excluding requests completed before s . We define

$$A_{\text{LAT}}(s, a) = L_{\text{singlebestscheduler}}(s) - L_{\text{CF}}(s, a), \quad H_{\text{LAT}}(s) = \max \left(0, \max_a A_{\text{LAT}}(s, a) \right), \quad (3)$$

where positive values reduce mean latency. The beneficial-state indicator is $B_{\text{LAT}}(s) = 1$ iff $\max_a A_{\text{LAT}}(s, a) > 0$. We report $P(B_{\text{LAT}} \mid D)$ and mean H_{LAT} separately from $P(D)$. The conceptual aggregate opportunity is therefore

$$P(D) \times P(B_{\text{LAT}} \mid D) \times \text{effect magnitude}, \quad (4)$$

not an exact deployable-performance equation. A high conditional benefit can coexist with a tiny overall opportunity.

The confirmatory mean uses a faithful-window clustered bootstrap with 2,000 replicates and seed 20260920. The cluster unit is the original fresh trace window, identified by workload source and window index, contributing disagreement states, so multiple states from one window do not become independent experimental units. Its verdict requires positive mean headroom and a clustered 95% lower endpoint above zero.

A resource-pressure regime is a workload window plus one frozen one-axis intervention, such as a KV-capacity cap, active-sequence cap, or arrival-rate scaling factor. “Support” and “opportunity” refer only to the existence of executable non-single best scheduler actions under that regime; usefulness requires the separate causal latency test above.

3.4. Objective audit

The historical objective is arrival-normalized weighted goodput,

$$\text{arrival} - \text{normalizedweightedgoodput} = \frac{\sum_i w_i \mathbf{1}[i \text{ completes by its SLO}]}{\sum_i w_i}. \quad (5)$$

The earlier’s all-terminal-arrival-normalized weighted goodput result is retained as an objective audit, not as fresh latency evidence. The latency study uses Eq. 3; its mean-latency result is confirmatory only for the fresh untouched causal universe.

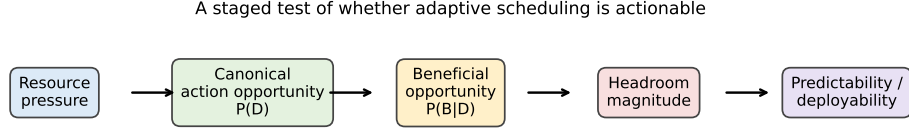


Figure 1: The paper’s evidence chain. A predictor is downstream of both action opportunity and causal value; deployment benefit is not assumed by the characterization.

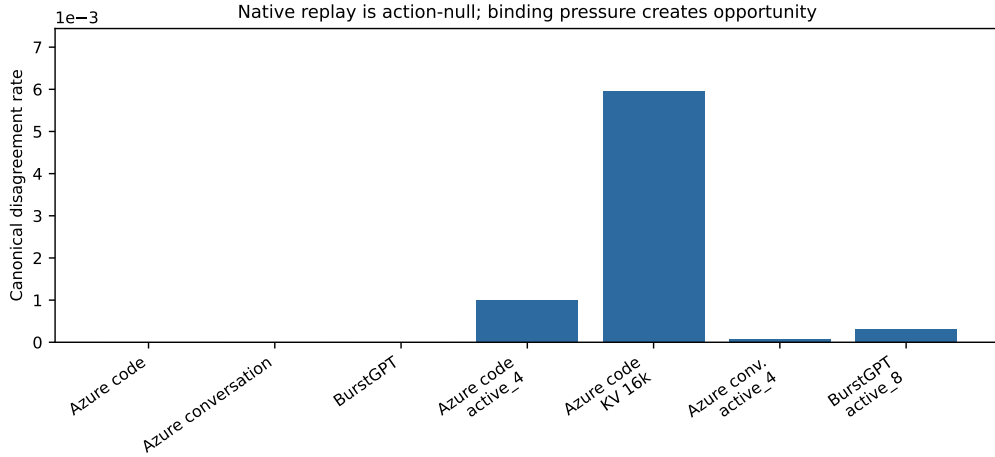


Figure 2: Native production-derived replay is action-null. Selected pressure conditions create nonzero canonical disagreement, with rates depending on workload and pressure axis. This is support for adaptive action selection, not evidence of selector benefit by itself.

4. Results: Native Action Opportunity

Under the frozen abundant-resource baseline, all three production-derived workloads were canonically action-null (Figure 1 summarizes the evidence chain; Figure 2 gives the rates). Azure code had 0 disagreements in 99,992 single best scheduler decision states; Azure conversation had 0/461,985; and BurstGPT had 0/440,461. The corresponding maximum KV utilizations were 0.002716, 0.001728, and 0.003802, with no active-cap binding states. Policy-ranking proxy differences existed in a small number of states, but canonical actions collapsed. This is not evidence that adaptive scheduling never matters; it is evidence that this resource regime offered no different executable six-policy action.

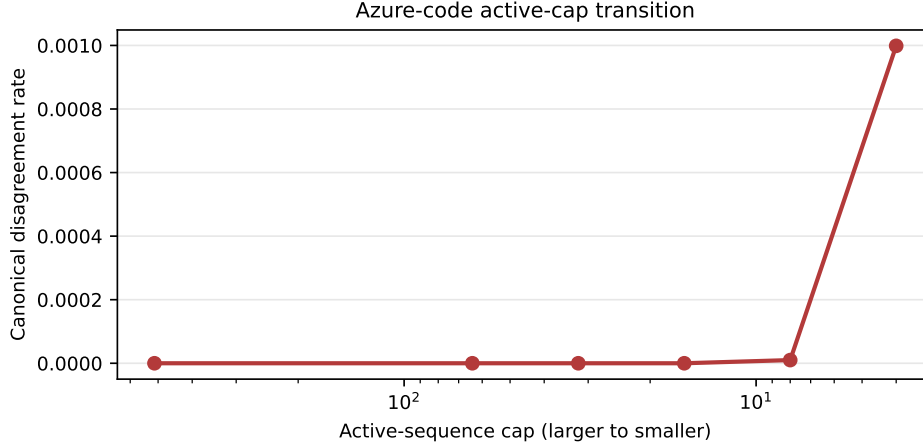


Figure 3: Azure-code active-sequence transition. Agreement persists until the active cap binds the trace, after which canonical disagreement increases.

5. Results: Resource-Regime Transition

The Resource-Regime map evaluated 1,080 one-axis conditions: 20 windows across three workload families, with active-sequence, KV-capacity, and arrival pressure axes (Figure 3; Table 2). There were 926 valid unconstrained, 38 partially constrained, 96 strongly constrained, and 20 horizon-invalid conditions. Across valid conditions, 11,328 canonical disagreement states were observed.

Arrival scaling through $8\times$ remained action-null for all three workload families. Active-sequence pressure created disagreement at active-cap 8 or 4 depending on workload; KV pressure created disagreement at the first binding or near-binding capacities. For Azure code, the first active-cap disagreement appeared at active-sequence cap = 8 and became sustained at active-sequence cap = 4. KV disagreement began at KV capacity = 16,000. Azure conversation first disagreed at active-sequence cap = 4 and showed KV disagreement at KV capacity = 16,000; BurstGPT showed active disagreement at active-sequence cap = 16, sustained at active-sequence cap = 8, and KV disagreement at KV capacity = 16,000.

The transition (Table 2) is the main regime result: pressure does not automatically create useful diversity, but concrete resource binding can turn a collapsed action set into a nontrivial one. In the studied traces, arrival-rate scaling alone did not create support through the tested range; tight KV

Table 2: Pressure-to-disagreement transition summary. “Sustained” means at least two windows with disagreement.

Workload	Axis	First binding	First disagreement	Sustained
Azure code	Active cap	active-sequence cap = 8	active-sequence cap = 8	active-sequence cap = 4
Azure code	KV cap	KV capacity = 16,000	KV capacity = 16,000	KV capacity = 16,000
Azure conversation	Active cap	active-sequence cap = 4	active-sequence cap = 4	active-sequence cap = 4
Azure conversation	KV cap	KV capacity = 8,000	KV capacity = 16,000	KV capacity = 8,000
BurstGPT	Active cap	active-sequence cap = 16	active-sequence cap = 16	active-sequence cap = 8
BurstGPT	KV cap	KV capacity = 16,000	KV capacity = 16,000	KV capacity = 16,000

capacity and active-sequence limits were the operative dimensions.

6. Results: Fresh Support and Causal Latency Headroom

The fresh support mapping used untouched windows and the frozen pressure protocol. The causal labeling plan selected five Azure code/conversation active/KV regimes and 720 disagreement states. BurstGPT contributed no selected causal regime, an important limitation rather than a missing result.

The confirmatory intervention found 590 beneficial states (Table 3; Figure 4), giving

$$P(B_{\text{LAT}} \mid D) = 590/720 = 0.8194444444.$$

Mean oracle latency headroom was 0.001995791 s (1.9958 ms), with a clustered 95% CI of [0.000190884, 0.003546196] s ([0.1909, 3.5462] ms). The frozen verdict was statistically supported. This is statistically supported but

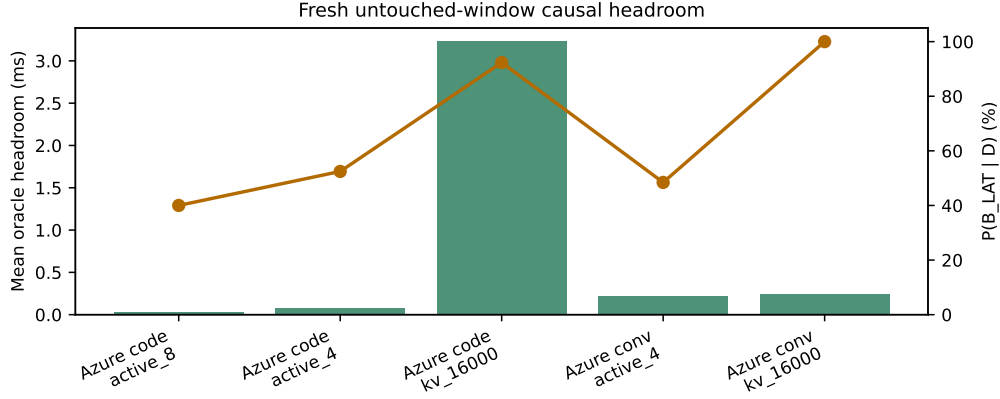


Figure 4: Fresh causal result by selected workload/regime. Bars show mean oracle headroom in milliseconds; the line shows $P(B_{LAT} | D)$. The oracle is a non-deployable opportunity bound over observed alternatives.

operationally modest. It establishes measurable local default-relative latency opportunity in the disagreement-support regime, not that the magnitude exceeds the engineering cost of adaptation. It is also not the claim that 81.9% of all scheduler decisions can be improved: $P(D)$ is often below one percent in the selected regimes and the conditional statistic is defined only on disagreement states.

6.1. Opportunity decomposition

Figure 5 makes the distinction between prevalence and value concrete. Azure-code KV capacity = 16,000 has the largest selected $P(D)$ and the largest mean headroom; active-cap regimes have much smaller $P(D)$ and correspondingly smaller aggregate opportunity even when some disagreement states are beneficial. A practitioner should therefore measure all three factors in Eq. 4: whether a different action exists, whether it helps conditional on existing, and whether its effect is large enough to matter relative to system noise and control overhead.

The practical reading is deliberately staged. Under abundant resources, this study observed no canonical disagreement and therefore no demonstrated action support for adaptive selection. Under constrained KV or active-sequence regimes, disagreement emerges and creates an executable opportunity. Within those disagreement states, fresh causal evidence shows frequent latency-beneficial alternatives. Opportunity and headroom still do not prove

Table 3: Fresh causal characterization. $P(D)$ uses all valid single best scheduler decision states; $P(B_{\text{LAT}} | D)$ uses only the disagreement states. Per-regime source-window counts report distinct fresh trace windows with at least one disagreement state in that regime. The same source window may contribute to multiple regimes, so the row counts are not additive; the 36 distinct workload-window pairs form the clusters used by the bootstrap.

Workload/regime	States	$P(D)$	$P(B_{\text{LAT}} D)$	Mean H_{LAT} (ms)	Source windows
Azure code, active-sequence cap = 8	10	0.000106	0.400	0.033	2
Azure code, active-sequence cap = 4	122	0.001290	0.525	0.077	15
Azure code, KV capacity = 16,000	431	0.004561	0.923	3.227	14
Azure conversation, active-sequence cap = 4	64	0.000108	0.484	0.217	17
Azure conversation, KV capacity = 16,000	93	0.000157	1.000	0.241	1
All selected fresh states	720	—	0.819	1.996	36 distinct

that a deployed adaptive selector is worthwhile.

7. Objective Sensitivity: arrival-normalized weighted goodput Saturation

The earlier The earlier pressure study completed with causal integrity and complete branch coverage, but all terminal arrival-normalized weighted goodput values were 1.0 and all arrival-normalized weighted goodput advantages were zero. An identifiability audit found no implementation bug: requests completed within deadlines and the objective saturated. Simple sensitivity cases confirmed that arrival-normalized weighted goodput distinguishes missed deadlines or unfinished work, but not different latency or queueing when all requests still meet a loose deadline.

Most counterfactual branches nevertheless changed latency. Thus zero arrival-normalized weighted goodput headroom does not imply zero latency

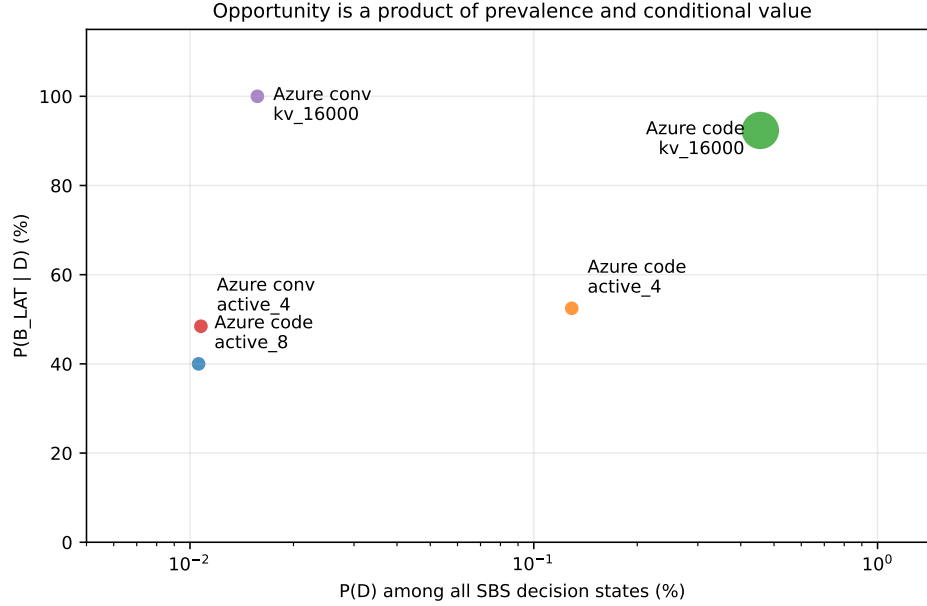


Figure 5: Practitioner regime map. Conditional benefit can be high while opportunity prevalence is small; point size is proportional to mean headroom.

headroom. We do not use the old The earlier post-hoc latency analysis as confirmatory evidence. The fresh untouched study in Section 6 is the confirmatory latency result, with its own pre-frozen primary endpoint and clustered bootstrap.

This is objective sensitivity, not evidence that arrival-normalized weighted goodput is invalid. arrival-normalized weighted goodput is an appropriate deadline-goodput objective for one question; mean latency is a different operational objective. A serving paper should not promote whichever metric yields the more favorable conclusion.

8. Bounded Real-vLLM Validation

The bounded real-vLLM validation used one RTX 5060 Ti, vLLM 0.27.1, Qwen2.5-0.5B-Instruct, and deterministic token-shape replay. It completed 40 full/chunked runs and a 20-run native chunked-prefill budget probe. Queueing and scheduler traces were retained. The verdict partially supported means support for the mechanism but not for the full causal or deployment

Table 4: Simulator-to-real-system correspondence. The real-system column supports pressure/action correspondence, not one-step causal headroom.

Property	Simulator/replay	Real vLLM
Low pressure	Canonical collapse	Low queue baseline
Resource binding	KV/active caps induce D	Queue and active-sequence pressure observed
Action difference	single best scheduler/six-policy canonical action	Native token-budget trace signature
Latency consequence	One-step single best scheduler-relative	Whole-run budget tradeoff
Exact intervention	Available	Not available

claim. Level 1 is demonstrated only in controlled replay/intervention: at disagreement states, changing the canonical action can expose one-step latency headroom. Level 2 is supported in real vLLM: resource pressure changes observable queue and scheduling behavior. Level 3 is not demonstrated: the paper does not show that a deployed adaptive selector on real vLLM would recover 1.9958 ms or improve end-to-end latency, SLOs, or throughput.

The native budget probe held chunked prefill fixed and varied the token budget. T512 produced 1,680 scheduler steps, 165 mixed prefill/decode steps, and 194 partial-prefill items. T4096 produced 1,536, 55, and 9. Waiting queues reached 7 and 5, respectively, while running sequences reached 4 in both cases. The low-load regime showed a 30.6 ms late-time-to-first-token difference and a 16.3 ms prompt-heavy end-to-end tradeoff; the high-load regime showed a smaller late-time-to-first-token difference and a 23.3 ms prompt-heavy end-to-end tradeoff.

The direct FULL-versus-CHUNKED comparison did not reproduce the simulator’s qualitative reversal (Table 4). KV utilization and preemption metrics were unavailable in the native probe because its metric parser returned empty dictionaries. The real result therefore supports the narrower statement that concrete serving pressure can alter observable scheduling behavior and latency. It is not real-system causal confirmation of Eq. 3.

9. Discussion

9.1. *When adaptation is justified*

The evidence does not support an anti-adaptive-scheduling conclusion. It supports a staging rule. Under abundant resources, a complex portfolio may collapse to the same action as single best scheduler, making adaptation overhead difficult to justify. Under binding KV or active-sequence pressure, executable alternatives can appear. Some fresh disagreement states contain positive latency headroom, but opportunity prevalence and effect magnitude remain regime-specific.

This matters because scheduler diversity is not scheduler value. A system that produces many distinct actions may still optimize the wrong objective; a system with high conditional benefit may have too few disagreement states to justify a router. The decomposition in Eq. 4 is a practical diagnostic, not a promise of deployment gain. The measured 1.9958 ms mean headroom is therefore evidence that causal opportunity exists in the sampled disagreement-support regime; deciding whether that opportunity exceeds selector latency, engineering complexity, and operational risk is a separate deployment question.

9.2. *Relation to modern adaptive systems*

LMetric, SOLA, Llumnix, MorphServe, Strata, Libra, FastServe, and related systems answer how to adapt serving behavior under dynamic resource and SLO conditions. ServeGen, BurstGPT, Azure traces, OpenTela, and longitudinal trace studies answer how to obtain or operate on realistic workload evidence. Our measurement asks when adaptation is unnecessary because canonical actions collapse, and when a local alternative is causally useful relative to a strong default. The questions are complementary; the present six-policy study does not claim to replace or exhaust modern schedulers, nor does the literature comparison substitute for a benchmark when making a direct performance claim.

9.3. *Limitations*

The supported scope is the studied production-derived trace families, the specific pressure regimes tested, the controlled replay/intervention framework, and the bounded real-vLLM configuration. The six-policy portfolio is not the universe of 2026 schedulers, and different portfolios may expose different support. BurstGPT did not contribute a selected fresh causal regime.

Fresh headroom is a mean-latency one-step oracle result, not a learned selector or production improvement; its mean effect is modest and may or may not exceed selector overhead. Fresh non-overlapping windows provide robustness within the trace families, not universal external validity. The real-vLLM validation uses one model, one RTX 5060 Ti GPU, vLLM 0.27.1, and one mechanism family, uses token-shape replay rather than exact private prompts, and supports the pressure/action mechanism only. It has no exact one-step real-system intervention, and the simulator-to-vLLM action mapping is imperfect. arrival-normalized weighted goodput saturation limits conclusions under that objective. Finally, we make no claim over all LLM-serving workloads, GPU architectures, model sizes, scheduler portfolios, serving systems, or forms of resource pressure; no claim of learned predictability, universal adaptive benefit, universal scheduler superiority, or impossibility; and no claim that measured causal headroom automatically justifies deployment.

10. Reproducibility and Artifact Scope

The public artifact (the public repository, see Data and Code Availability) records trace provenance, frozen manifests, support grids, fresh non-overlap proofs, causal state and action manifests, source hashes, bootstrap seed 20260920, and the real-vLLM environment. The principal result packages are Phase A native replay, Phase B pressure mapping, fresh confirmatory causal latency, and real-vLLM pressure/action validation. Historical selector and joint-240 artifacts remain available but are not treated as evidence of learned deployment readiness.

11. Conclusion

Adaptive LLM serving should be evaluated in stages. Faithful production-derived replay can be action-null under abundant resources; resource pressure can create canonical executable alternatives; and fresh one-step replay can show that some alternatives contain positive local mean-latency headroom. In this study, the fresh confirmation found 590 beneficial states among 720 disagreement states. Mean oracle headroom was 1.9958 ms, with a positive corrected clustered confidence interval. The result is conditional and modest: the conditional benefit probability does not equate to overall deployment value.

Terminal arrival-normalized weighted goodput saturation demonstrates why objective choice matters, while bounded vLLM evidence provides real-system support for the pressure mechanism and observable action differentiation without claiming real causal headroom or deployment value. This performance-evaluation methodology shows that adaptation should be justified by measured opportunity and causal value; closed-loop and deployed value evaluation remains future work.

Declaration of generative AI and AI-assisted technologies in the manuscript preparation process

During the preparation of this work, the author used ChatGPT by OpenAI, OpenAI Codex, and Google Gemini in order to assist with organization, language refinement, literature-search planning, software development, and workflow support. After using these tools, the author reviewed and edited the content as needed and takes full responsibility for the content of the publication.

Declaration of Competing Interest

The author declares that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgements

The author gratefully acknowledges Professor Ioannis Koutis for his research guidance and scientific feedback. The author thanks the NJIT Wulver HPC for providing computational resources for the fresh confirmatory computation, and Anders Borum (Secure ShellFish) for providing tools that enabled remote research access. Finally, the author thanks his mother for her continued support.

Funding and Support

This work received in-kind computational support through the Google Cloud Research Credits Program (USD 1,000 in cloud credits) and computational/tooling support from CloudRift Inc. Neither Google nor CloudRift

Inc. had any role in the study design, data collection, analysis, interpretation, manuscript preparation decisions, or the decision to submit the article for publication. No monetary research grant was received from these organizations.

Data and Code Availability

Code, derived research artifacts, experiment configurations, and reproducibility artifacts for this study are available at <https://github.com/SoroushVahidi/llm-serving-heuristic-evolution>. Raw third-party workload traces are obtained from their original upstream sources. The derived research data and reproducibility package have been permanently archived at DOI: <https://doi.org/10.5281/zenodo.22865294>.

References

- [1] J. R. Rice, The algorithm selection problem, *Advances in Computers* 15 (1976) 65–118.
- [2] C. P. Gomes, B. Selman, Algorithm portfolios, *Artificial Intelligence* 126 (1–2) (2001) 43–62.
- [3] B. Sun, Z. Huang, H. Zhao, W. Xiao, X. Zhang, Y. Li, W. Lin, Llumnix: Dynamic scheduling for large language model serving, in: *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*, USENIX Association, 2024, pp. 173–191.
- [4] K. Hong, X. Li, L. Chen, Q. Mao, G. Dai, X. Ning, S. Yan, Y. Liang, Y. Wang, SOLA: Optimizing SLO attainment for large language model serving with state-aware scheduling, in: *Proceedings of Machine Learning and Systems*, Vol. 7, 2025.
- [5] Y. Zhong, S. Liu, J. Chen, J. Hu, Y. Zhu, X. Liu, X. Jin, H. Zhang, Dist-Serve: Disaggregating prefill and decoding for goodput-optimized large language model serving, in: *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*, USENIX Association, 2024, pp. 193–210.

- [6] D. Zhang, J. Han, K. Zhang, X. Wei, S. Shen, C. Fang, W. Yu, J. Zhou, R. Chen, LMetric: Simple is better – multiplication may be all you need for LLM request scheduling, in: 20th USENIX Symposium on Operating Systems Design and Implementation (OSDI 26), USENIX Association, 2026.
- [7] T. Choudhury, G. Joshi, W. Wang, Job assignment in machine learning inference systems with accuracy constraints, *Performance Evaluation* 167 (2025) 102463. doi:10.1016/j.peva.2024.102463.
- [8] A. Ali, X. Ma, S. Zawad, P. Aditya, I. E. Akkus, R. Chen, L. Yang, F. Yan, Enabling scalable and adaptive machine learning training via serverless computing on public cloud, *Performance Evaluation* 167 (2025) 102451. doi:10.1016/j.peva.2024.102451.
- [9] Microsoft Azure, Azure LLM inference trace 2023, <https://github.com/Azure/AzurePublicDataset/blob/master/AzureLLMInferenceDataset2023.md>, public LLM inference trace dataset released by Azure (2023).
- [10] Y. Wang, Y. Chen, Z. Li, X. Kang, Y. Fang, Y. Zhou, Y. Zheng, Z. Tang, X. He, R. Guo, X. Wang, Q. Wang, A. C. Zhou, X. Chu, BurstGPT: A real-world workload dataset to optimize LLM serving systems, in: *Proceedings of the 31st ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, ACM, 2025, pp. 5831–5841. doi:10.1145/3711896.3737413.
- [11] W. Kwon, Z. Li, S. Zhuang, Y. Sheng, L. Zheng, C. H. Yu, J. E. Gonzalez, H. Zhang, I. Stoica, Efficient memory management for large language model serving with PagedAttention, in: *Proceedings of the 29th Symposium on Operating Systems Principles (SOSP 2023)*, ACM, 2023, pp. 611–626.
- [12] G.-I. Yu, J. S. Jeong, G.-W. Kim, S. Kim, B.-G. Chun, Orca: A distributed serving system for transformer-based generative models, in: *16th USENIX Symposium on Operating Systems Design and Implementation (OSDI 22)*, USENIX Association, 2022, pp. 521–538.
- [13] A. Agrawal, N. Kedia, A. Panwar, J. Mohan, N. Kwatra, B. S. Gulanani, A. Tumanov, R. Ramjee, Taming throughput-latency tradeoff in

- LLM inference with Sarathi-Serve, in: 18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24), USENIX Association, 2024, pp. 117–134.
- [14] P. Patel, E. Choukse, C. Zhang, A. Shah, I. Goiri, S. Maleki, R. Bianchini, Splitwise: Efficient generative LLM inference using phase splitting, in: Proceedings of the 51st Annual International Symposium on Computer Architecture, IEEE Computer Society, 2024. doi:10.1109/ISCA59077.2024.00019.
 - [15] B. Wu, Y. Zhong, Z. Zhang, S. Liu, F. Liu, Y. Sun, G. Huang, X. Liu, X. Jin, FastServe: Iteration-Level preemptive scheduling for large language model inference, in: 23rd USENIX Symposium on Networked Systems Design and Implementation (NSDI 26), USENIX Association, 2026, pp. 57–74.
 - [16] C. Ruan, Y. Chen, D. Tian, Y. Shi, Y. Wu, J. Li, C. Li, Libra: Flexible request partitioning and scheduling for serving unbalanced and dynamic LLM workloads, in: 23rd USENIX Symposium on Networked Systems Design and Implementation (NSDI 26), USENIX Association, 2026.
 - [17] R. Qin, Z. Li, W. He, J. Cui, F. Ren, M. Zhang, Y. Wu, W. Zheng, X. Xu, Mooncake: Trading more storage for less computation – a KVCache-centric architecture for serving LLM chatbot, in: 23rd USENIX Conference on File and Storage Technologies (FAST 25), USENIX Association, 2025, pp. 155–170.
 - [18] Z. Xie, Z. Xu, M. Zhao, Y. An, V. S. Mailthody, S. Mahlke, M. Garland, C. Kozyrakis, Strata: Hierarchical context caching for long context language model serving, arXiv preprint arXiv:2508.18572 (2025).
 - [19] Z. Su, T. Lan, Z. Wang, J. Yang, Y. Cheng, Efficient and workload-aware LLM serving via runtime layer swapping and KV cache resizing, arXiv preprint arXiv:2506.02006 (2025).
 - [20] J. Stojkovic, C. Zhang, I. Goiri, J. Torrellas, E. Choukse, DynamoLLM: Designing LLM inference clusters for performance and energy efficiency, in: Proceedings of the 2025 IEEE International Symposium on High Performance Computer Architecture (HPCA 25), IEEE, 2025, pp. 1348–1362. doi:10.1109/HPCA61900.2025.00102.

- [21] W. Nixon, J. Durbin, F. Standhartinger, H. S. Gunawi, J. Yang, A year in LLM serving: Workload evolution, caching and load-balancing, arXiv preprint arXiv:2608.13573 (2026).
- [22] Y. Xiang, X. Li, K. Qian, W. Yu, E. Zhai, X. Jin, ServeGen: Workload characterization and generation of large language model serving in production, in: 23rd USENIX Symposium on Networked Systems Design and Implementation (NSDI 26), USENIX Association, 2026.
- [23] X. Yao, Y. Jiang, I. Badanin, Q. Hu, R. M. Smith, B. Yuan, I. Schlag, E. Yoneki, A. Klimovic, OpenTela: Unifying decentralized computing resources for heterogeneous LLM serving, in: 20th USENIX Symposium on Operating Systems Design and Implementation (OSDI 26), USENIX Association, 2026, pp. 1821–1838.
- [24] L. Xu, F. Hutter, H. H. Hoos, K. Leyton-Brown, SATzilla: Portfolio-based algorithm selection for SAT, *Journal of Artificial Intelligence Research* 32 (2008) 565–606.
- [25] M. Lindauer, H. H. Hoos, F. Hutter, T. Schaub, AutoFolio: An automatically configured algorithm selector, *Journal of Artificial Intelligence Research* 53 (2015) 745–778.